

02 · REST: paginación (descargar la colección completa)

Ninguna API entrega todos los registros en una sola respuesta. La API de tienda-virtual pagina **por número de página**: cada lista devuelve `pageInfo { page, pageSize, total, totalPages }`. Hay dos formas de recorrerla entera:

- **Forma A** — pedir `page` 1, 2, 3... y parar cuando `page == totalPages`.
- **Forma B** — leer `total` de la primera respuesta y calcular `totalPages = ceil(total / pageSize)` de antemano.

Otras APIs pagan **por cursor**: en vez de un número de página te devuelven un puntero (`next`) a la página siguiente y se itera hasta que vale `null`. El bucle es el mismo; solo cambia "incrementa `page` " por "sigue el enlace `next` ".

```
import csv
import math
import os

import requests

BASE = "http://localhost:3000"
API_KEY = "sk_demo_00000000000000000000000000000000"
TIMEOUT = 30

sesion = requests.Session()
sesion.headers.update({"x-api-key": API_KEY, "User-Agent": "MineriaWeb-2026-2/1.0"})

DATA_DIR = os.path.join(os.getcwd(), "..", "data")
os.makedirs(DATA_DIR, exist_ok=True)
```

Forma A — bucle por número de página

`descargar_todo` sirve para cualquier listado paginado de esta API (`productos`, `clientes`, `ordenes`, `testimonios`).

```
def descargar_todo(path, page_size=50, **filtros):
    items = []
    page = 1
    while True:
        r = sesion.get(
            f"{BASE}{path}",
            params={**filtros, "page": page, "pageSize": page_size},
            timeout=TIMEOUT,
        )
        r.raise_for_status()
        payload = r.json()
        items.extend(payload["items"])
```

```

        info = payload["pageInfo"]
        print(f"  {path}  página {info['page']:>2}/{info['totalPages']}  (+
{len(payload['items'])}, total {len(items)}/{info['total']})")
        if info["page"] >= info["totalPages"]:
            return items
        page += 1

```

```

productos = descargar_todo("/api/productos")
clientes = descargar_todo("/api/clientes", page_size=40)
print(f"\n{len(productos)} productos, {len(clientes)} clientes")

```

```

/api/productos  página  1/2  (+50, total 50/90)
/api/productos  página  2/2  (+40, total 90/90)
/api/clientes   página  1/3  (+40, total 40/120)
/api/clientes   página  2/3  (+40, total 80/120)
/api/clientes   página  3/3  (+40, total 120/120)

```

90 productos, 120 clientes

Forma B — calcular las páginas de antemano

Se pide la primera página solo para leer `total`, y con eso ya sabemos cuántas peticiones faltan (útil para barras de progreso o para paralelizar).

```

PAGE_SIZE = 25
primera = sesion.get(
    f"{BASE}/api/productos", params={"page": 1, "pageSize": PAGE_SIZE},
    timeout=TIMEOUT
).json()

total = primera["pageInfo"]["total"]
total_paginas = math.ceil(total / PAGE_SIZE)
print(f"total={total}  pageSize={PAGE_SIZE}  ->  {total_paginas} páginas")

todos = list(primera["items"])
for page in range(2, total_paginas + 1):
    todos.extend(
        sesion.get(
            f"{BASE}/api/productos", params={"page": page, "pageSize": PAGE_SIZE},
            timeout=TIMEOUT
        ).json()["items"]
    )
print(f"descargados {len(todos)} productos en {total_paginas} peticiones")

```

```

total=90  pageSize=25  ->  4 páginas
descargados 90 productos en 4 peticiones

```

No todo se pagina: GET /api/comentarios

Algunos endpoints devuelven **todos** los registros que coinciden con el filtro, sin `pageInfo`. Aquí, todos los comentarios de un producto.

```

r = sesion.get(
    f"{BASE}/api/comentarios",
    params={"tipo": "producto", "productoId": 1},

```

```

        timeout=TIMEOUT,
    )
    r.raise_for_status()
    data = r.json()
    print("Claves de la respuesta:", list(data.keys()), "(sin pageInfo)")
    print(f"{data['total']} comentarios del producto 1")
    for c in data["items"][:3]:
        print(f"    {c['calificacion']}* {c['clienteNombre']} {c['clienteApellidos']}: {c['texto'][:60]}...")

```

Claves de la respuesta: ['items', 'total'] (sin pageInfo)

4 comentarios del producto 1

4* Nicolas David Perez Salazar: Compre el iPhone 15 Pro Max para trabajo y la fluidez de iOS...

3* Daniela Fiorella Ochoa Vidal: Excelente camara pero el precio sigue siendo dificil de just...

2* Luciana Valentina Torres Mejia: El precio me parece excesivo para el salto de funciones fren...

Guardar en data/rest_productos.csv y data/rest_clientes.csv

```

ruta_productos = os.path.join(DATA_DIR, "rest_productos.csv")
with open(ruta_productos, "w", newline="", encoding="utf-8") as f:
    writer = csv.DictWriter(
        f,
        fieldnames=["id", "codigo", "nombre", "categoria", "subcategoria", "precio", "stock", "rating_promedio", "total_resenas"],
    )
    writer.writeheader()
    for p in productos:
        ar = p["aggregateRating"] or {}
        writer.writerow({
            "id": p["id"], "codigo": p["codigo"], "nombre": p["nombre"],
            "categoria": p["categoria"], "subcategoria": p["subcategoria"],
            "precio": p["precio"], "stock": p["stock"],
            "rating_promedio": ar.get("ratingValue", ""),
            "total_resenas": ar.get("reviewCount", ""),
        })

ruta_clientes = os.path.join(DATA_DIR, "rest_clientes.csv")
with open(ruta_clientes, "w", newline="", encoding="utf-8") as f:
    writer = csv.DictWriter(
        f, fieldnames=["id", "dni", "nombre", "apellidos", "email", "ciudad", "pais", "fecha_registro"]
    )
    writer.writeheader()
    for c in clientes:
        writer.writerow({k: c.get(k, "") for k in writer.fieldnames})

print("Guardado:", ruta_productos)
print("Guardado:", ruta_clientes)

```

Guardado: /private/tmp/claude-501/-Users-ferris-pucp-mineria-mineria-web-2026-2/1b003788-da84-40ac-86e0-5b0e2134c0b1/scratchpad/build/repo/sesion-de-clase-03/notebooks/./data/rest_productos.csv

Guardado: /private/tmp/claude-501/-Users-ferris-pucp-mineria-mineria-web-2026-2/1b003788-da84-40ac-86e0-5b0e2134c0b1/scratchpad/build/repo/sesion-de-clase-03/notebooks/./data/rest_clientes.csv

Hasta aquí la API key fue "un header más". En el siguiente notebook la **seguridad de la API** es el tema: qué responde ante credenciales ausentes o inválidas, qué son los *scopes* y cómo se revoca una clave.